

05 - Hashing

Bu bölümde, sözlükleri uygulamanın çok verimli bir yolunu ele alıyoruz. Bir sözlüğün, elemanları üzerinde arama (lookup), ekleme ve silme işlemleri tanımlanmış bir soyut veri türü olduğunu hatırlayın. Bu kümenin elemanları keyfi bir doğaya sahip olabilir: sayılar, bir alfabenin karakterleri, karakter dizileri vb. Pratikte en önemli durum, kayıtların (bir okuldaki öğrenci kayıtları, bir devlet dairesindeki vatandaş kayıtları, bir kütüphanedeki kitap kayıtları) olduğu durumdur.

Genellikle kayıtlar, kaydın temsil ettiği varlık hakkında belirli bir türde bilgi tutan birkaç alandan oluşur. Örneğin, bir öğrenci kaydı, öğrencinin kimlik numarası, adı, doğum tarihi, cinsiyeti, ev adresi, ana dalı vb. alanlar içerebilir. Kayıt alanları arasında genellikle kayıtların temsil ettiği varlıkları tanımlamak için kullanılan en az bir *anahtar* (key) bulunur (örneğin, öğrencinin kimlik numarası).

$K_1, K_2, \dots, K_n$ anahtarlarına sahip n kayıttan oluşan bir sözlük uygulamamız gerektiğini varsayıyoruz.

Hashleme, anahtarları *hash tablosu* adı verilen tek boyutlu bir dizi $H[0..m-1]$ arasında dağıtma fikrine dayanır. Dağıtım, her bir anahtar için önceden tanımlanmış bir fonksiyon olan *hash fonksiyonu* h 'nin değerini hesaplayarak yapılır. Bu fonksiyon, bir anahtara 0 ile $m-1$ arasında bir tamsayı atar, buna *hash adresi* denir.

Örneğin, anahtarlar negatif olmayan tamsayılar ise, bir hash fonksiyonu $h(K) = K \bmod m$ şeklinde olabilir; açıkça, m 'ye bölmenin kalanı her zaman 0 ile $m-1$ arasındadır. Anahtarlar bir alfabenin harfleri ise, önce bir harfe alfabedeki konumunu atayabiliriz (burada $ord(K)$ olarak gösterilir) ve ardından tamsayılar için kullanılan aynı türde bir fonksiyon uygulayabiliriz. Son olarak, K bir karakter dizisi $c_0 c_1 \dots c_{s-1}$ ise, çok basit bir seçenek olarak $(\sum_{i=0}^{s-1} ord(c_i)) \bmod m$ kullanabiliriz. Daha iyi bir seçenek ise $h(K)$ 'yi şu şekilde hesaplamaktır:

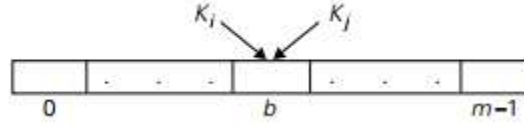
$$h \leftarrow 0; \text{for } i \leftarrow 0 \text{ to } s-1 \text{ do } h \leftarrow (h * C + ord(c_i)) \bmod m,$$

burada C , her $ord(c_i)$ 'den büyük bir sabittir.

Genel olarak, bir hash fonksiyonunun birbiriyle çelişen bazı gereksinimleri karşılaması gerekir:

- Bir hash tablosunun boyutu, anahtar sayısına kıyasla aşırı derecede büyük olmamalı, ancak uygulamanın zaman verimliliğini tehlikeye atmamak için yeterli olmalıdır (aşağıya bakın).
- Bir hash fonksiyonunun, anahtarları hash tablosunun hücreleri arasında mümkün olduğunca eşit şekilde dağıtması gerekir. (Bu gereksinim, çoğu uygulama için, bir hash fonksiyonunun bir anahtarın tüm bitlerine bağlı olmasını, yalnızca bazılarına değil, arzu edilir kılar.)
- Bir hash fonksiyonunun hesaplanması kolay olmalıdır.

Elbette, eğer hash tablosunun boyutunu m , anahtarların sayısı olan n 'den daha küçük seçersek, *çarpışmalar* (*collisions*) meydana gelir (Şekil 1)—yani iki veya daha fazla anahtarın hash tablosunda aynı hücreye yerleştirilmesi. Ancak çarpışmalar, m , n 'den önemli ölçüde büyük olsa bile beklenmelidir. Hatta en kötü durumda, tüm anahtarlar hash tablosunda aynı hücreye denk gelebilir. Neyse ki, uygun seçilmiş bir hash tablo büyüklüğü ve iyi bir hash fonksiyonu ile bu durum çok nadir gerçekleşir. Yine de her hash yönteminin bir çarpışma çözümleme mekanizmasına ihtiyacı vardır. Bu mekanizma, hashing yönteminin iki temel türünde farklıdır: açık hashing (*open hashing*) (ayrı zincirleme - *separate chaining* olarak da adlandırılır) ve kapalı hashing (*closed hashing*) (açık adresleme - *open addressing* olarak da adlandırılır).



Şekil 1. Hashing'de iki anahtarın çarpışması: $h(K_i) = h(K_j)$

Açık Hashing (Ayrı Zincirleme)

Açık hashing yönteminde anahtarlar, hash tablosundaki hücelere bağlanmış olan bağlantılı listelerde (linked lists) saklanır. Her liste, kendi hücrelerine hash fonksiyonu tarafından eşlenmiş tüm anahtarları içerir. Örneğin, aşağıdaki kelime listesini ele alalım:

A, FOOL, AND, HIS, MONEY, ARE, SOON, PARTED.

Hash fonksiyonu olarak, yukarıda belirtilen kelimeler için basit bir fonksiyon kullanacağız. Bu fonksiyon, kelimedeki harflerin alfabadeki sıralamalarının toplamını hesaplayıp, sonucun 13 ile bölümünden kalanını bulmak üzerine kuruludur.

Boş bir tabloyla başlıyoruz. İlk anahtar kelime **A**'dır; hash değeri:

$$h(A) = 1 \bmod 13 = 1$$

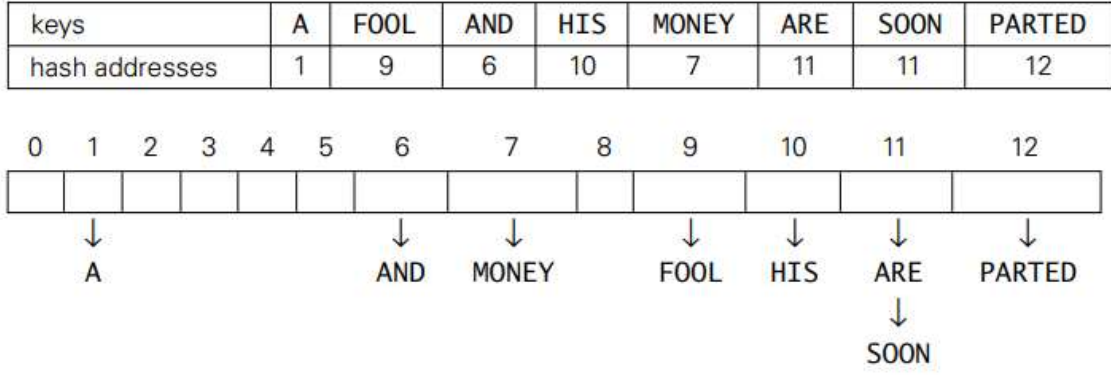
İkinci anahtar kelime olan **FOOL**, dokuzuncu hücreye yerleştirilir:

$$(6 + 15 + 15 + 12) \bmod 13 = 9$$

Bu işlemi tüm kelimeler için sürdürdüğümüzde, işlemin son hali Şekil 2'de gösterilmiştir. Bu örnekte, **ARE** ve **SOON** kelimelerinin çarpıştığına dikkat edin. Çünkü:

$$h(ARE) = (1 + 18 + 5) \bmod 13 = 11 \text{ ve } h(SOON) = (19 + 15 + 15 + 14) \bmod 13 = 11$$

Bağlantılı listelerden oluşan böyle bir tabloda arama nasıl yapılır? Arama işlemini, tabloyu oluştururken kullanılan yöntemle aynısını uygulayarak gerçekleştiririz.



Şekil 2. Hash anahtar değerleri ve hash adresleri (üstte); Açık Hash tablosu (altta).

Örneğin, yukarıda verilen tabloda **KID** kelimesini aramak istediğimizi varsayalım. İlk olarak hash fonksiyonunu bu anahtar için hesaplarız:

$$h(KID) = 11$$

11 numaralı hücreye bağlı olan liste boş değildir, dolayısıyla aranan kelime bu listede olabilir. Fakat olası çarpışmalar sebebiyle, tüm listeyi tarayıp karşılaştırma yapmadan emin olamayız. Bu örnekte önce **ARE**, ardından **SOON** kelimeleriyle karşılaştırırız ve aramanın başarısız olduğunu anlarız.

Genel olarak aramanın verimliliği, bağlantılı listelerin uzunluğuna bağlıdır. Bu uzunluklar ise sözlükteki anahtarların sayısına, hash tablosunun boyutuna ve hash fonksiyonunun kalitesine bağlıdır.

Eğer hash fonksiyonu, n adet anahtarı hash tablosundaki m hücreye eşit şekilde dağıtırsa, her listenin uzunluğu yaklaşık olarak n/m olacaktır. Buradaki oran, yani $\alpha = n/m$, hash tablosunun **yük faktörü (load factor)** olarak adlandırılır ve hashing verimliliği açısından kritik bir rol oynar. Özellikle, başarılı ve başarısız aramalarda ortalama incelenen işaretçi (zincir bağlantıları) sayıları sırasıyla:

$$S \approx 1 + \frac{\alpha}{2} \text{ ve } U = \alpha$$

şeklinde ifade edilir.

Bu sonuçlar, rastgele seçilen bir elemanın aranması ve anahtarların tablo hücrelerine düzgün şekilde dağıldığı varsayımı altında elde edilmiştir. Bu sonuçlar oldukça doğaldır. Gerçekte, neredeyse doğrudan bağlantılı liste içinde ardışık arama yapmakla aynıdır; hashing kullanarak elde ettiğimiz fayda ise, ortalama liste boyutunu hash tablosunun boyutu m oranında azaltmamızdır.

Genellikle, yük faktörünün 1 değerinden çok uzak olmamasını isteriz. Çok küçük bir yük faktörü, çok sayıda boş liste ve dolayısıyla verimsiz alan kullanımı anlamına gelir. Çok büyük bir yük faktörü ise daha uzun bağlantılı listeler ve dolayısıyla daha uzun arama süreleri yaratır. Ancak yük faktörünü yaklaşık 1 tutarsak, oldukça verimli bir yapı elde ederiz. Böylece belirli bir anahtarı ortalama olarak yalnızca bir veya iki karşılaştırmayla bulabiliriz! Tabii ki karşılaştırmaların yanı sıra, arama için hash fonksiyonunu hesaplamak da zaman alacaktır, ancak bu işlem n ve m 'den bağımsız, sabit zamanlı bir

işlemdir. Bu yöntemle elde ettiğimiz bu yüksek verimlilik, yöntem zekasının ve ayrıca ekstra alan kullanımının bir sonucudur.

Sözlükteki diğer işlemler olan ekleme ve silme, arama işlemine neredeyse tamamen benzerdir. Ekleme işlemleri normalde listenin sonuna yapılır. Silme işlemi ise silinecek anahtarın önce listede aranması, ardından listeden çıkarılması şeklinde gerçekleştirilir. Dolayısıyla bu işlemlerin verimlilikleri, aramanın verimliliğiyle aynıdır. Eğer anahtarların sayısı n , hash tablosunun büyüklüğü m 'ye yaklaşık eşitse, tüm bu işlemlerin ortalama durumdaki zaman karmaşıklığı $\theta(1)$ 'dir.

Kapalı Hashing (Açık Adresleme)

Kapalı hashing yönteminde tüm anahtarlar, bağlantılı listeler kullanmadan doğrudan hash tablosunun hücrelerinde saklanır. (Bu durumda, tablo boyutunun mmm anahtar sayısı nnn'ye en az eşit olması gerektiği açıktır.) Çarpışmaları çözmek için farklı stratejiler uygulanabilir. Bunlardan en basiti, **lineer yoklama (linear probing)** adı verilen yöntemdir. Bu yöntemde çarpışmanın gerçekleştiği hücreden hemen sonraki hücre kontrol edilir. Eğer bu hücre boşsa, yeni anahtar buraya yerleştirilir. Hücre doluysa, bir sonraki hücreye bakılır ve bu işlem boş bir hücre bulunana kadar devam eder. Tablo sonuna ulaşırsa, arama tablonun başına dönerek devam eder; yani tablo dairesel bir dizi gibi değerlendirilir. Bu yöntem, aynı kelime listesi ve hash fonksiyonuyla birlikte aşağıda gösterilmiştir.

keys	A	FOOL	AND	HIS	MONEY	ARE	SOON	PARTED
hash addresses	1	9	6	10	7	11	11	12

	0	1	2	3	4	5	6	7	8	9	10	11	12
		A											
		A								FOOL			
		A					AND			FOOL			
		A					AND			FOOL	HIS		
		A					AND	MONEY		FOOL	HIS		
		A					AND	MONEY		FOOL	HIS	ARE	
		A					AND	MONEY		FOOL	HIS	ARE	SOON
PARTED		A					AND	MONEY		FOOL	HIS	ARE	SOON

Şekil 3. Doğrusal problema ile hash tablosu oluşturma örneği.

Verilen bir K anahtarını aramak için önce tablonun oluşturulmasında kullanılan hash fonksiyonuyla $h(K)$ değerini hesaplarız. Eğer $h(K)$ 'nin gösterdiği hücre boşsa, arama başarısızdır. Eğer hücre doluysa, K 'yı hücredeki değerle karşılaştırırız; eşleşiyorlarsa anahtarı bulmuş oluruz. Eşleşme yoksa, sonraki hücreye geçerek aynı işlemi eşleşme ya da boş bir hücre bulana kadar sürdürürüz. Örneğin, yukarıdaki tabloda **LIT** kelimesini ararsak:

$$h(LIT) = (12 + 9 + 20) \bmod 13 = 2$$

2 numaralı hücre boş olduğu için hemen arama durur. Ancak **KID** kelimesini arıyorsak:

$$h(KID) = (11 + 9 + 4) \bmod 13 = 11$$

Bu durumda önce 11 numaralı hücredeki **ARE**, sonra **SOON**, ardından **PARTED** ve **A** anahtarlarıyla karşılaştırmalar yaparız. Bunların hiçbiri uyuşmadığından ve sonrasında boş bir hücreye ulaştığımız için, aramanın başarısız olduğunu belirtiriz.

Arama ve ekleme işlemleri kapalı hashing için oldukça basittir; ancak silme işlemi biraz karmaşıktır. Örneğin, eğer yukarıda verilen hash tablosundan **ARE** kelimesini doğrudan silersek, daha sonra aynı bölgede bulunan **SOON** kelimesini bulamayız. Çünkü, arama işlemi boş bir hücre gördüğünde başarısız olarak sonlandırılır. Bu sorunun basit bir çözümü "tembel silme (lazy deletion)" adı verilen yöntemdir; yani daha önce dolu olan ancak silinen hücreleri özel bir sembolle işaretleyerek, hiç kullanılmamış hücrelerden ayırt ederiz.

Lineer yoklama yönteminin matematiksel analizi, ayrı zincirleme yöntemine göre çok daha zordur. Bu analizlerin basitleştirilmiş sonuçları, yük faktörü α ile hash tablosundaki başarılı ve başarısız aramalarda algoritmanın erişmesi gereken ortalama hücre sayısının sırasıyla şu şekilde olduğunu belirtmektedir:

$$S \approx \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right) \text{ and } U \approx \frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right)$$

(ve bu yaklaşımların doğruluğu, hash tablosunun boyutu büyüdükçe artar). İlginç bir şekilde, bu sayılar, yüksek yük faktörü (α 'nın büyük değerleri) olan yoğun dolu tablolarda bile şaşırtıcı derecede küçüktür.

α	$\frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right)$	$\frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right)$
50%	1.5	2.5
75%	2.5	8.5
90%	5.5	50.5

Yine de hash tablosu doluluğa yaklaştıkça, lineer yoklama yönteminin performansı **kümeleşme (clustering)** olarak adlandırılan bir durum nedeniyle kötüleşir. Lineer yoklamada bir **küme (cluster)**, birbirini takip eden dolu hücrelerin oluşturduğu dizidir (tablonun sonundan başına doğru döngü olabilir). Örneğin, Şekil 3'de verilen tablonun son halinde iki ayrı küme bulunmaktadır. Kümeler hashing için olumsuz bir durumdur, çünkü sözlük işlemlerini verimsiz hale getirirler. Kümeler büyüdükçe, yeni bir elemanın kümeye dahil edilme ihtimali artar; ayrıca büyük kümeler, yeni anahtar eklenirken iki kümenin birleşme ihtimalini artırarak daha fazla kümeleşmeye yol açar.

Bu sorunu hafifletmek için başka birçok çarpışma çözümleme stratejisi önerilmiştir. Bunlardan en önemlilerinden biri **çift hashing (double hashing)** yöntemidir. Bu yöntemde, $l = h(K)$ konumunda bir çarpışma meydana geldiğinde kullanılacak problama dizisini belirlemek için ikinci bir hash fonksiyonu $s(K)$ kullanılır ve bu fonksiyon sabit bir artış miktarını sağlar:

$$(l + s(K)) \bmod m, (l + 2s(K)) \bmod m, \dots$$

Tablodaki her konumun (Şekil 3) dizisi tarafından yoklanmasını garanti etmek için artış miktarı olan $s(k)$ ile tablo boyutu m 'nin aralarında asal olması gerekir, yani tek ortak bölenleri 1 olmalıdır. (Eğer m sayı zaten asal ise bu koşul otomatik olarak sağlanır.) Literatürde önerilen bazı fonksiyonlar şunlardır: Küçük tablolar için $s(k) = m - 2 - k \bmod (m - 2)$ ve $s(k) = 8 - (k \bmod 8)$ büyük tablolar için ise $s(k) = k \bmod 97 + 1$.

Çift hashing yönteminin matematiksel analizi oldukça zordur. Bu konuda elde edilen kısmi teorik sonuçlar ve pratik deneyimler, hem birincil hem de ikincil hash fonksiyonları iyi seçildiği takdirde çift hashing yönteminin lineer yoklamadan daha iyi olduğunu göstermektedir. Fakat çift hashing yönteminin performansı da, tablo doluluğa yaklaştıkça kötüleşir. Bu durumda doğal çözüm, **yeniden hashing (rehashing)** işlemidir: Mevcut tablo taranır ve tüm anahtarlar daha büyük boyutlu yeni bir tabloya yerleştirilir.

Hashing yöntemini sözlüklerin uygulanmasında temel rakibi olan dengeli arama ağaçlarıyla karşılaştırmak yararlı olacaktır. Bu iki yöntemin temel özellikleri şöyledir:

- **Asimptotik Zaman Verimliliği:** Hashing yönteminde arama, ekleme ve silme işlemleri ortalama olarak $\theta(1)$ zamanda gerçekleşirken, oldukça nadir gerçekleşen en kötü durumda $\theta(n)$ zamanda tamamlanır. Dengeli arama ağaçlarında ise hem ortalama hem de en kötü durumda zaman karmaşıklığı $\theta(\log n)$ 'dir.
- **Sıralama Özelliğinin Korunması:** Dengeli arama ağaçlarının aksine, hashing yöntemi anahtarlar arasında belirli bir sıralama varsayımı yapmaz ve genellikle anahtarların sıralamasını korumaz. Bu özellik, hashing yöntemini anahtarları sıralı bir şekilde gezmek ya da belirli alt ve üst sınırlar arasındaki anahtarları saymak gibi aralığa bağlı sorguların gerektiği uygulamalar için daha az uygun hale getirir.

1950'lerde IBM araştırmacıları tarafından keşfedilmesinden bu yana hashing, birçok önemli uygulama alanında kullanılmıştır. Özellikle programların derlenmesi sırasında oluşturulan sembol tablolarını (programdaki sembollerin listesini tutan tablolar) depolamak için standart bir teknik haline gelmiştir. Ayrıca hashing, satranç oynayan bilgisayar programları tarafından oluşturulan pozisyonların daha önce değerlendirilip değerlendirilmediğini kontrol etmek gibi yapay zekâ uygulamalarında da oldukça kullanışlıdır. Bazı değişikliklerle, disk üzerindeki büyük sözlükleri saklamak için de faydalı olduğu kanıtlanmıştır; hashing'in bu varyasyonu **genişletilebilir hashing (extendible hashing)** olarak adlandırılır. Disk erişimi, ana bellekte gerçekleştirilen yoklamalara göre daha maliyetli olduğundan, disk erişimlerinin mümkün olduğunca az sayıda yapılması tercih edilir. Bu nedenle, genişletilebilir hashing yönteminde hash fonksiyonunun hesapladığı konum, diskteki ve bbb sayıda anahtar tutabilen bir **kova (bucket)**'nın adresini belirtir. Bir anahtarın hangi kovada olduğu belirlendikten sonra, kovadaki tüm anahtarlar ana belleğe okunur ve aranan anahtar bu küme içerisinde aranır.